

BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 2 Practical No: 5

Roll no: 49 PRN No: 12414235 Name of Student: Saloni Tiwari

Subject Name: Deep Learning

Problem Statement:

Implement and compare RNN, LSTM, and GRU models for sequence classification, and analyze their performance using appropriate evaluation metrics.

Algorithm:

- **Import Libraries:** Import TensorFlow/Keras, NumPy, Pandas, Matplotlib, and Scikit-learn for data processing, model creation, training, evaluation, and visualization.
- **Load Dataset:** Load the AAPL stock price dataset containing Date, Open, High, Low, Close, and Volume values.
- **Explore Dataset:** Display the dataset shape, columns, sample records, and visualize the historical closing prices.
- **Preprocess Data:** Select the closing price, handle missing values, and normalize the values between 0 and 1 using MinMaxScaler.
- **Create Time-Series Sequences:** Use the previous 60 trading days to predict the closing price of the next day.
- **Split the Data:** Divide the sequential data into 80% training and 20% testing sets while maintaining chronological order.
- **Create LSTM Model:** Build an LSTM network with two LSTM layers of 50 units, Dropout layers, and a Dense output layer.
- **Compile the Model:** Use the Adam optimizer and Mean Squared Error loss function.
- **Train the Model:** Train the model for 20 epochs with a batch size of 32 and validate using the test data.
- **Analyze Training Performance:** Plot training and validation loss to analyze model learning and overfitting.
- **Perform Prediction:** Predict closing prices for the test sequences and convert them back to the original price scale.
- **Evaluate the Model:** Calculate MAE and RMSE to measure prediction accuracy.
- **Plot Prediction Performance:** Plot actual and predicted closing prices for comparison.

Code:

```
1] # Importing the required libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout

2] # Loading and exploring the AAPL stock dataset
url = "https://raw.githubusercontent.com/kristoffer-paulsson/stock-data/master/aapl.csv"
data = pd.read_csv(url, sep=";")
data["date"] = pd.to_datetime(data["date"])
print("Dataset shape:", data.shape)
print("Columns:", data.columns.tolist())
print(data.head())

3] # Visualizing the historical closing price
plt.figure(figsize=(12, 5))
plt.plot(data["date"], data["close"])
plt.title("AAPL Stock Closing Price")
plt.xlabel("Date")
plt.ylabel("Closing Price")
plt.grid()
plt.show()

4] # Preprocessing and normalizing the closing prices
prices = data[["close"]].dropna().values
scaler = MinMaxScaler(feature_range=(0, 1))
scaled_prices = scaler.fit_transform(prices)
print("Number of prices:", len(prices))
print("Normalized data shape:", scaled_prices.shape)

5] # Creating time-series sequences using the previous 60 days
X = []
y = []
for i in range(60, len(scaled_prices)):
    X.append(scaled_prices[i-60:i, 0])
    y.append(scaled_prices[i, 0])
X = np.array(X)
y = np.array(y)
X = X.reshape(X.shape[0], X.shape[1], 1)
print("X shape:", X.shape)
print("y shape:", y.shape)

6] # Splitting the data into training and testing sets
split = int(len(X) * 0.8)
X_train = X[:split]
X_test = X[split:]
y_train = y[:split]
y_test = y[split:]
print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))

7] # Creating the LSTM neural network
model = Sequential([
    LSTM(50, return_sequences=True, input_shape=(60, 1)),
    Dropout(0.2),
    LSTM(50),
    Dropout(0.2),
    Dense(1)
])
model.compile(
    optimizer="adam",
    loss="mean_squared_error"
```

```

)
model.summary()

8] # Training the LSTM model
history = model.fit(
    X_train,
    y_train,
    epochs=20,
    batch_size=32,
    validation_data=(X_test, y_test),
    verbose=1
)

9] # Plotting training and validation loss
plt.figure(figsize=(10, 5))
plt.plot(
    history.history["loss"],
    label="Training Loss"
)
plt.plot(
    history.history["val_loss"],
    label="Validation Loss"
)
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Training vs Validation Loss")
plt.legend()
plt.grid()
plt.show()

10] # Generating predictions on the test data
predictions = model.predict(X_test)
predictions = scaler.inverse_transform(predictions)
actual = scaler.inverse_transform(
    y_test.reshape(-1, 1)
)
print("Predictions generated successfully")

11] # Evaluating the LSTM model using MAE and RMSE
mae = mean_absolute_error(
    actual,
    predictions
)
rmse = np.sqrt(
    mean_squared_error(
        actual,
        predictions
    )
)
print("Mean Absolute Error (MAE):", mae)
print("Root Mean Squared Error (RMSE):", rmse)

12] # Plotting actual vs predicted stock prices
plt.figure(figsize=(12, 5))
plt.plot(
    actual,
    label="Actual Price"
)
plt.plot(
    predictions,
    label="Predicted Price"
)
plt.xlabel("Trading Days")
plt.ylabel("Price")
plt.title("AAPL Stock Price: Actual vs Predicted")
plt.legend()
plt.grid()
plt.show()

13] # Predicting the next closing price

```

```

last_60_days = scaled_prices[-60:]
last_60_days = last_60_days.reshape(
    1, 60, 1
)
next_price = model.predict(
    last_60_days
)
next_price = scaler.inverse_transform(
    next_price
)
print(
    "Predicted next closing price: $",
    round(float(next_price[0][0]), 2)
)

```

Output:

1] IMDB dataset loaded and reviews padded to a fixed sequence length.

```

# Load the IMDB movie review dataset
max_features = 10000
maxlen = 200

(x_train, y_train), (x_test, y_test) = imdb.load_data(
    num_words=max_features
)

# Make all sequences the same length
x_train = pad_sequences(x_train, maxlen=maxlen)
x_test = pad_sequences(x_test, maxlen=maxlen)

print("Training data shape:", x_train.shape)
print("Testing data shape:", x_test.shape)
print("Training labels shape:", y_train.shape)
print("Testing labels shape:", y_test.shape)

```

```

Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb.npz
17464789/17464789 ————— 0s 0us/step
Training data shape: (25000, 200)
Testing data shape: (25000, 200)
Training labels shape: (25000,)
Testing labels shape: (25000,)

```

```

# Check the number of positive and negative reviews
print("Negative reviews:", np.sum(y_train == 0))
print("Positive reviews:", np.sum(y_train == 1))

```

```

Negative reviews: 12500
Positive reviews: 12500

```

2] Common model architecture defined for the three sequence classification models.

```
# Build RNN, LSTM, or GRU model
def build_model(kind):
    model = Sequential([
        Embedding(max_features, 128)
    ])

    if kind == "RNN":
        model.add(SimpleRNN(64))
    elif kind == "LSTM":
        model.add(LSTM(64))
    elif kind == "GRU":
        model.add(GRU(64))

    model.add(Dense(1, activation="sigmoid"))

    model.compile(
        optimizer="adam",
        loss="binary_crossentropy",
        metrics=["accuracy"]
    )

    return model
```

3] RNN, LSTM and GRU trained for three epochs with validation data.

```
# Train the Simple RNN model
rnn_model = build_model("RNN")

rnn_history = rnn_model.fit(
    x_train,
    y_train,
    epochs=3,
    batch_size=128,
    validation_split=0.2,
    verbose=1
)
```

Epoch 1/3
157/157 ————— 8s 30ms/step - accuracy: 0.6772 - loss: 0.5774 - val_accuracy: 0.8184 - val_loss: 0.4223
Epoch 2/3
157/157 ————— 3s 20ms/step - accuracy: 0.8578 - loss: 0.3410 - val_accuracy: 0.8380 - val_loss: 0.4023
Epoch 3/3
157/157 ————— 3s 18ms/step - accuracy: 0.9360 - loss: 0.1743 - val_accuracy: 0.7876 - val_loss: 0.5846

```
# Train the LSTM model
lstm_model = build_model("LSTM")

lstm_history = lstm_model.fit(
    x_train,
    y_train,
    epochs=3,
    batch_size=128,
    validation_split=0.2,
    verbose=1
)
```

Epoch 1/3
157/157 ————— 7s 22ms/step - accuracy: 0.7736 - loss: 0.4552 - val_accuracy: 0.8686 - val_loss: 0.3320
Epoch 2/3
157/157 ————— 2s 14ms/step - accuracy: 0.9030 - loss: 0.2460 - val_accuracy: 0.8598 - val_loss: 0.3211
Epoch 3/3
157/157 ————— 2s 14ms/step - accuracy: 0.9384 - loss: 0.1702 - val_accuracy: 0.8658 - val_loss: 0.3269

```
# Train the GRU model
gru_model = build_model("GRU")

gru_history = gru_model.fit(
    x_train,
    y_train,
    epochs=3,
    batch_size=128,
    validation_split=0.2,
    verbose=1
)
```

Epoch 1/3
157/157 ————— 4s 17ms/step - accuracy: 0.7474 - loss: 0.4895 - val_accuracy: 0.8290 - val_loss: 0.3840
Epoch 2/3
157/157 ————— 3s 16ms/step - accuracy: 0.8942 - loss: 0.2653 - val_accuracy: 0.8636 - val_loss: 0.3282
Epoch 3/3
157/157 ————— 4s 26ms/step - accuracy: 0.9263 - loss: 0.1937 - val_accuracy: 0.8448 - val_loss: 0.3603

4] Classification reports and confusion matrices generated for the sequence models.

```
[11] # Evaluate all three models
rnn_result, rnn_cm = evaluate_model(rnn_model, "RNN")
lstm_result, lstm_cm = evaluate_model(lstm_model, "LSTM")
gru_result, gru_cm = evaluate_model(gru_model, "GRU")

...

RNN Classification Report
-----
              precision    recall  f1-score   support

   Negative       0.83       0.73       0.78     12500
   Positive       0.76       0.85       0.80     12500

 accuracy              0.79     25000
 macro avg              0.80     25000
weighted avg              0.80     25000

Confusion Matrix:
[[ 9162  3338]
 [ 1884 10616]]

LSTM Classification Report
-----
              precision    recall  f1-score   support

   Negative       0.87       0.85       0.86     12500
   Positive       0.85       0.88       0.87     12500

 accuracy              0.86     25000
 macro avg              0.86     25000

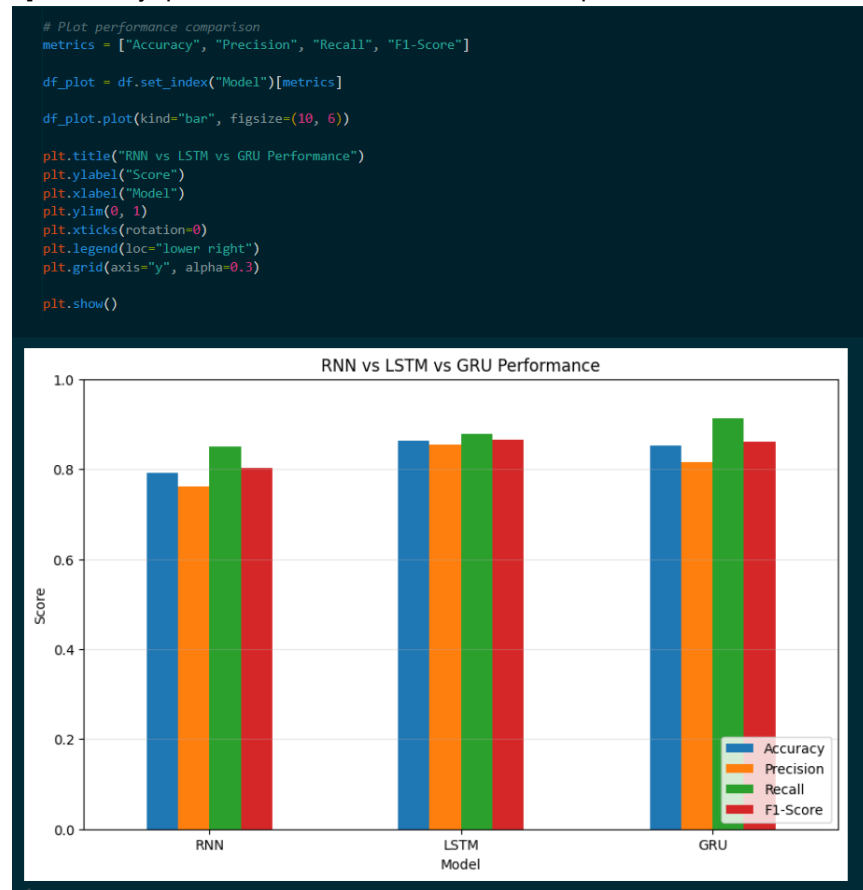
...

Confusion Matrix:
[[ 9921  2579]
 [ 1094 11406]]

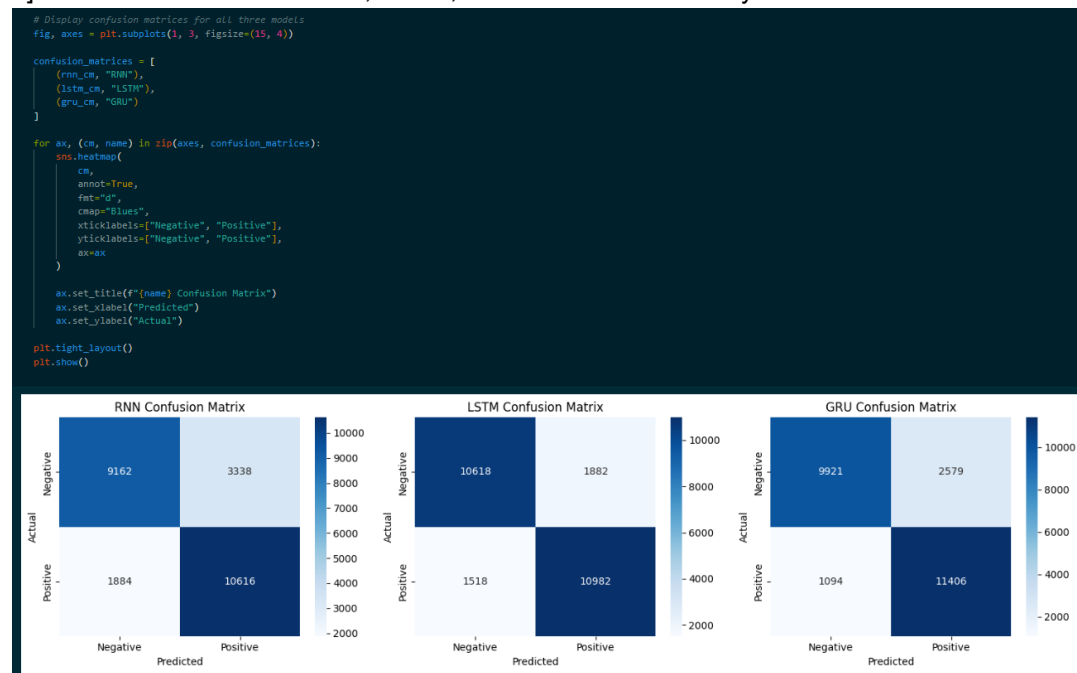
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

	Model	Accuracy	Precision	Recall	F1-Score
0	RNN	0.7911	0.7608	0.8493	0.8026
1	LSTM	0.8640	0.8537	0.8786	0.8660
2	GRU	0.8531	0.8156	0.9125	0.8613

5] Accuracy, precision, recall, and F1-score compared across RNN, LSTM, and GRU.



6] Confusion matrices of RNN, LSTM, and GRU shown side-by-side.



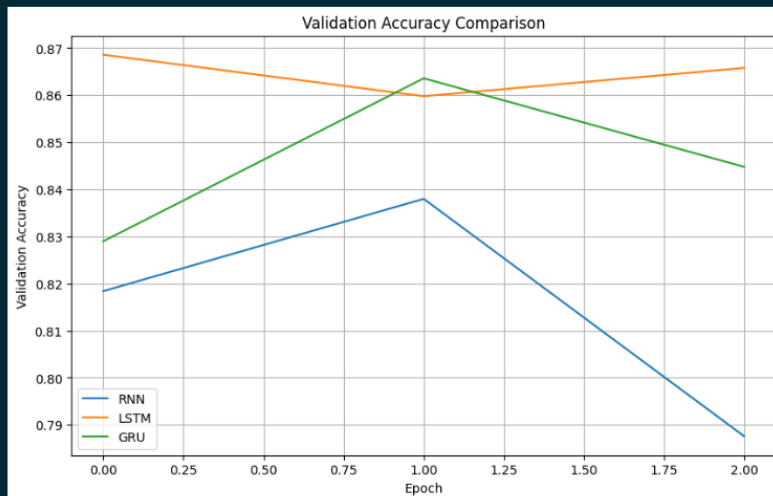
7] Validation accuracy trends of the three models across training epochs.

```
# Compare training and validation accuracy
plt.figure(figsize=(10, 6))

plt.plot(rnn_history.history["val_accuracy"], label="RNN")
plt.plot(lstm_history.history["val_accuracy"], label="LSTM")
plt.plot(gru_history.history["val_accuracy"], label="GRU")

plt.title("Validation Accuracy Comparison")
plt.xlabel("Epoch")
plt.ylabel("Validation Accuracy")
plt.legend()
plt.grid(True)

plt.show()
```



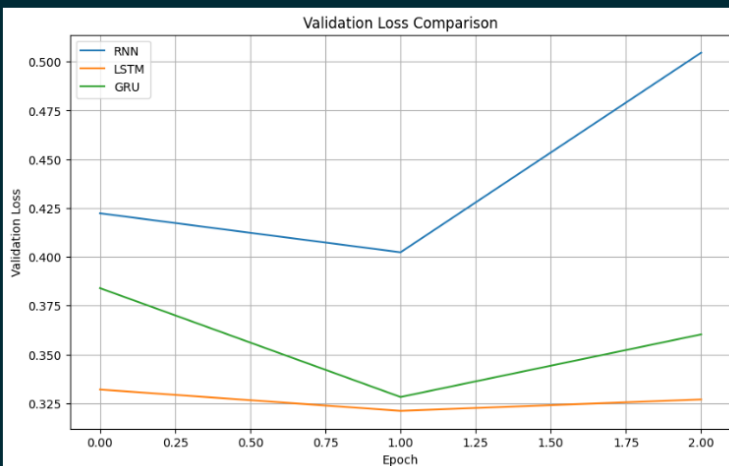
8] Validation loss trends showing the learning behavior of each model.

```
# Compare validation loss
plt.figure(figsize=(10, 6))

plt.plot(rnn_history.history["val_loss"], label="RNN")
plt.plot(lstm_history.history["val_loss"], label="LSTM")
plt.plot(gru_history.history["val_loss"], label="GRU")

plt.title("Validation Loss Comparison")
plt.xlabel("Epoch")
plt.ylabel("Validation Loss")
plt.legend()
plt.grid(True)

plt.show()
```



9] Maximum observed accuracy, precision, recall, and F1-score identified for model comparison.

```
# Display the highest observed value for each metric
for metric in ["Accuracy", "Precision", "Recall", "F1-Score"]:
    best_value = df[metric].max()
    models = df.loc[df[metric] == best_value, "Model"].tolist()

    print(f"{metric}: {best_value:.4f} ({', '.join(models)})")
```

Accuracy: 0.8640 (LSTM)

Precision: 0.8537 (LSTM)

Recall: 0.9125 (GRU)

F1-Score: 0.8660 (LSTM)